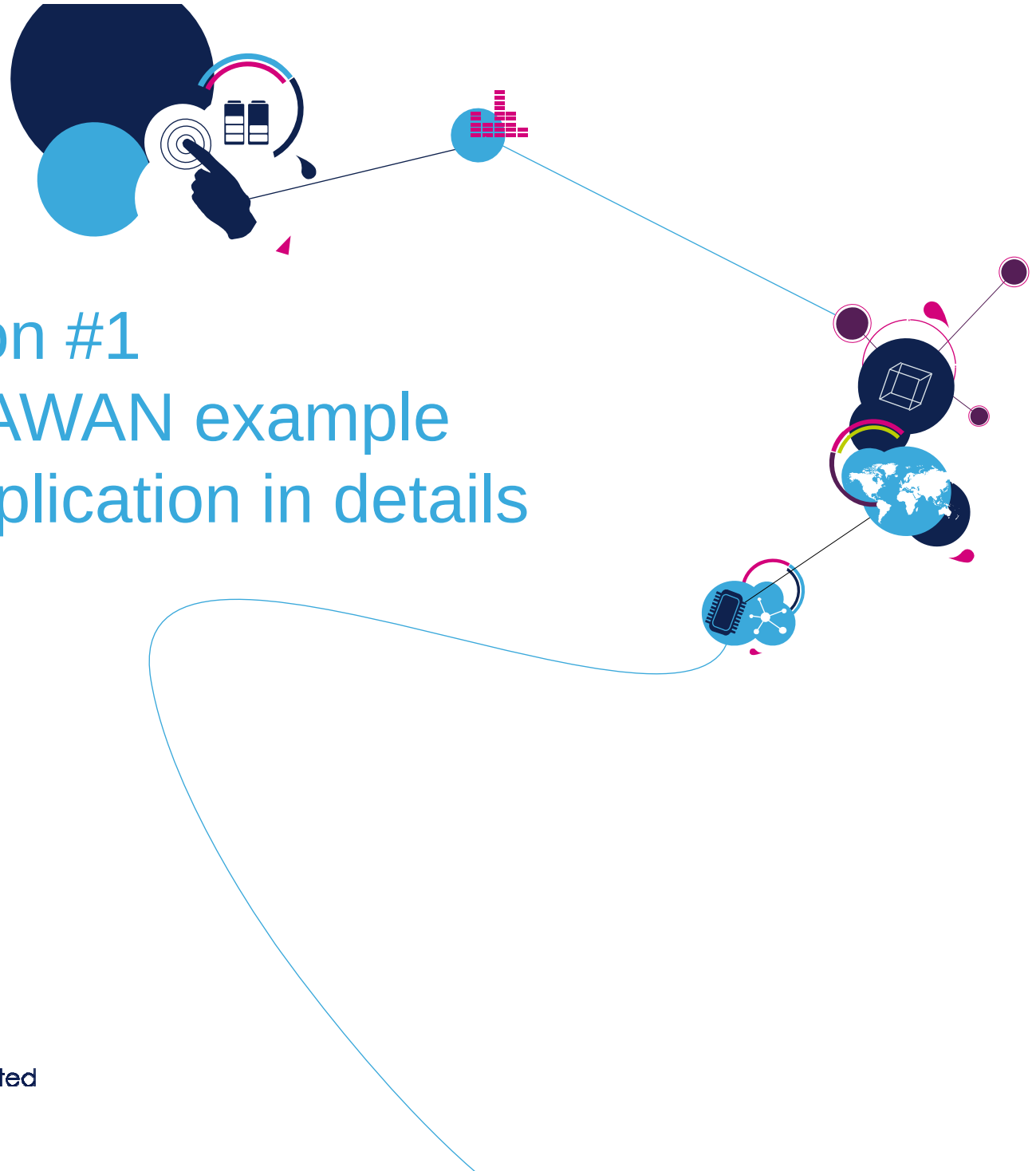


LoRa hands-on #1

I-CUBE-LORAWAN example

“AT-Slave” application in details



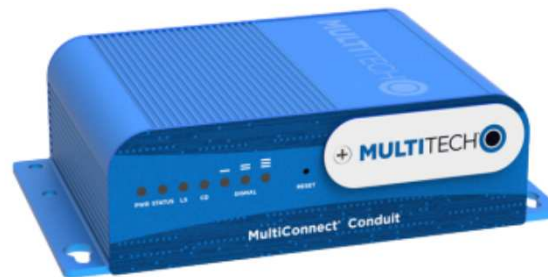
Hands-on

- Key learning

- Configuration of gateway and network server
- Application server basics and functionality
- I-CUBE-LRWAN software package: “AT_Slave” project file structure as example of LoRa class A device project framework
- IEEE 802.15.4 introduction
- Operational model of LoRa application layer
- LoRa protocol stack implementation details (parametrization, software timers, hw functions, low power mode, debug)

Hands-on

Gateway / Network / Application server



Hands-on

LoRa gateway/network server configuration

LoRa Configuration [Show Advanced Settings](#)

Enabled	☒	Mode	NETWORK SERVER
Frequency Band	868	Public	☒
Channel Plan	EU868	Lease Time	60-00-00 dd-hh-mm
Additional Channels	869.5	Network ID	EUI
Tx Power (dBm)	10	EUI	01:01:01:01:01:01:01:01
Antenna Gain	0	Network Key	Key
Rx 1 DR Offset	0	Key	2B:7E:15:16:28:AE:D2
Rx 2 Datarate	12	NetID	000000
Address Range Start	00:00:00:01	Duty Cycle Period	60
Address Range End	FF:FF:FF:FE	Adr Step	30
Queue Size	1	Min Datarate	0
		Max Datarate	4

Network Server Logging

Log Destination	SYSLOG
Path	/var/log/
Log Level	MAXIMUM

Network Server Testing

Disable Join Rx1	☐
Disable Join Rx2	☐
Disable Rx1	☐
Disable Rx2	☐
Disable Duty Cycle	☒

868MHz ISM band

To avoid delayed response

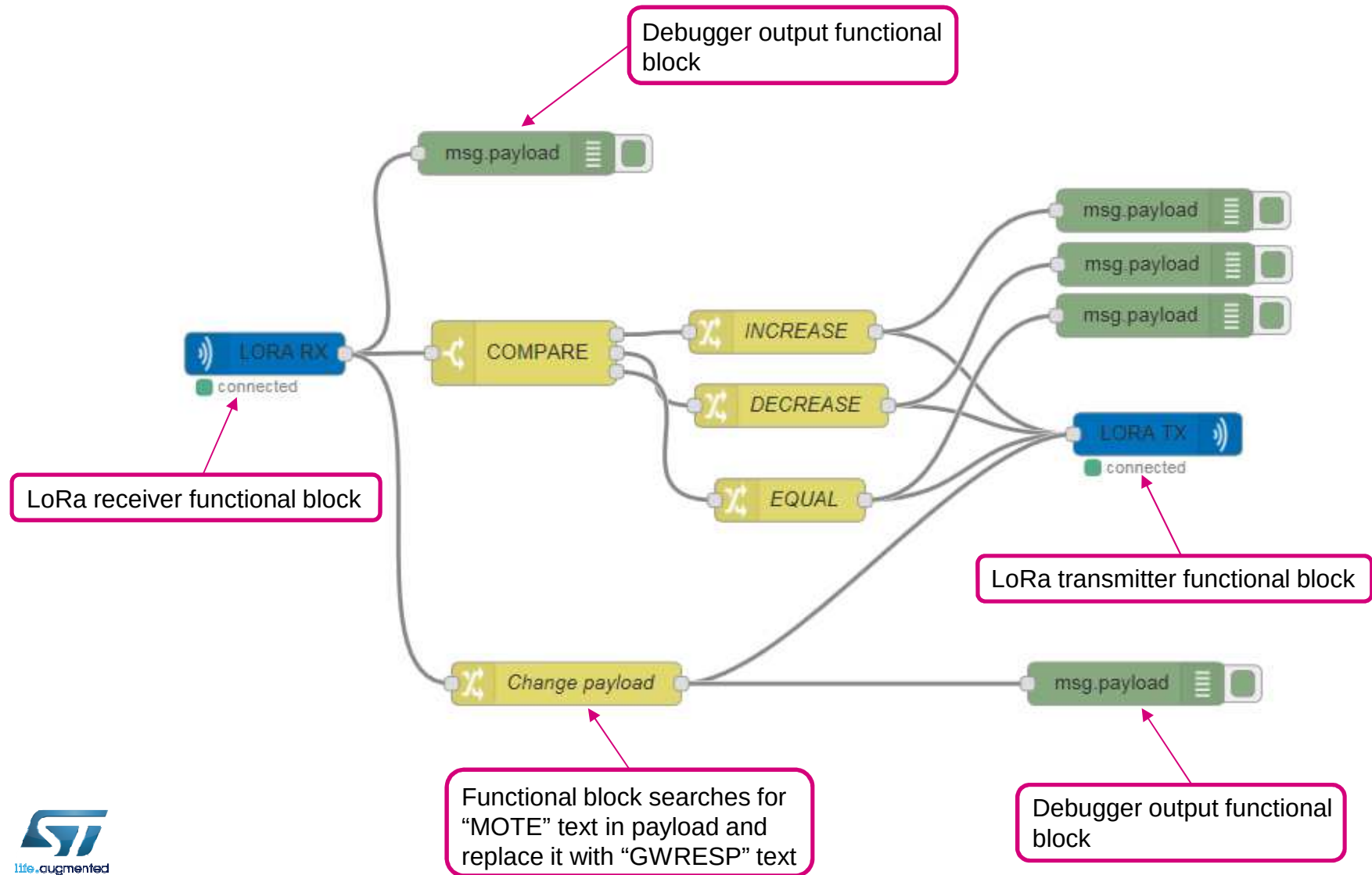
According to Comissioning.h

During development only

 **life.augmented**

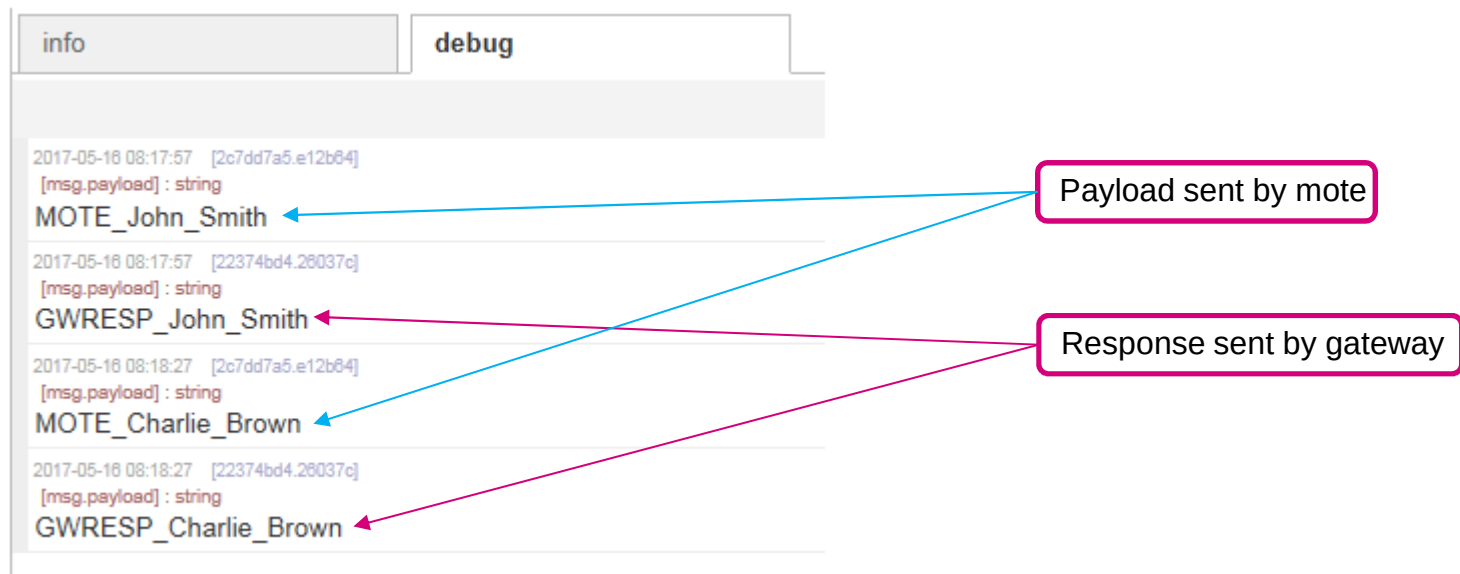
Hands-on

LoRa application server (Node-RED application functionality)



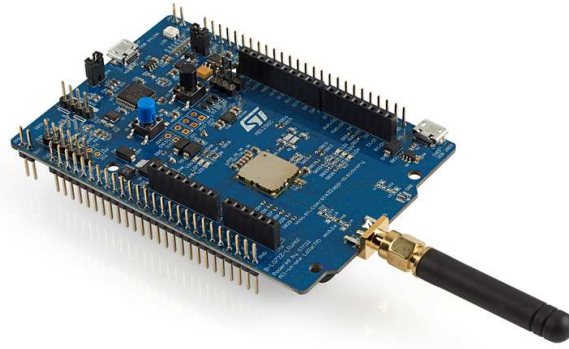
Hands-on

LoRa application server (Node-RED application debug output)



Hands-on

I-CUBE-LRWAN software package

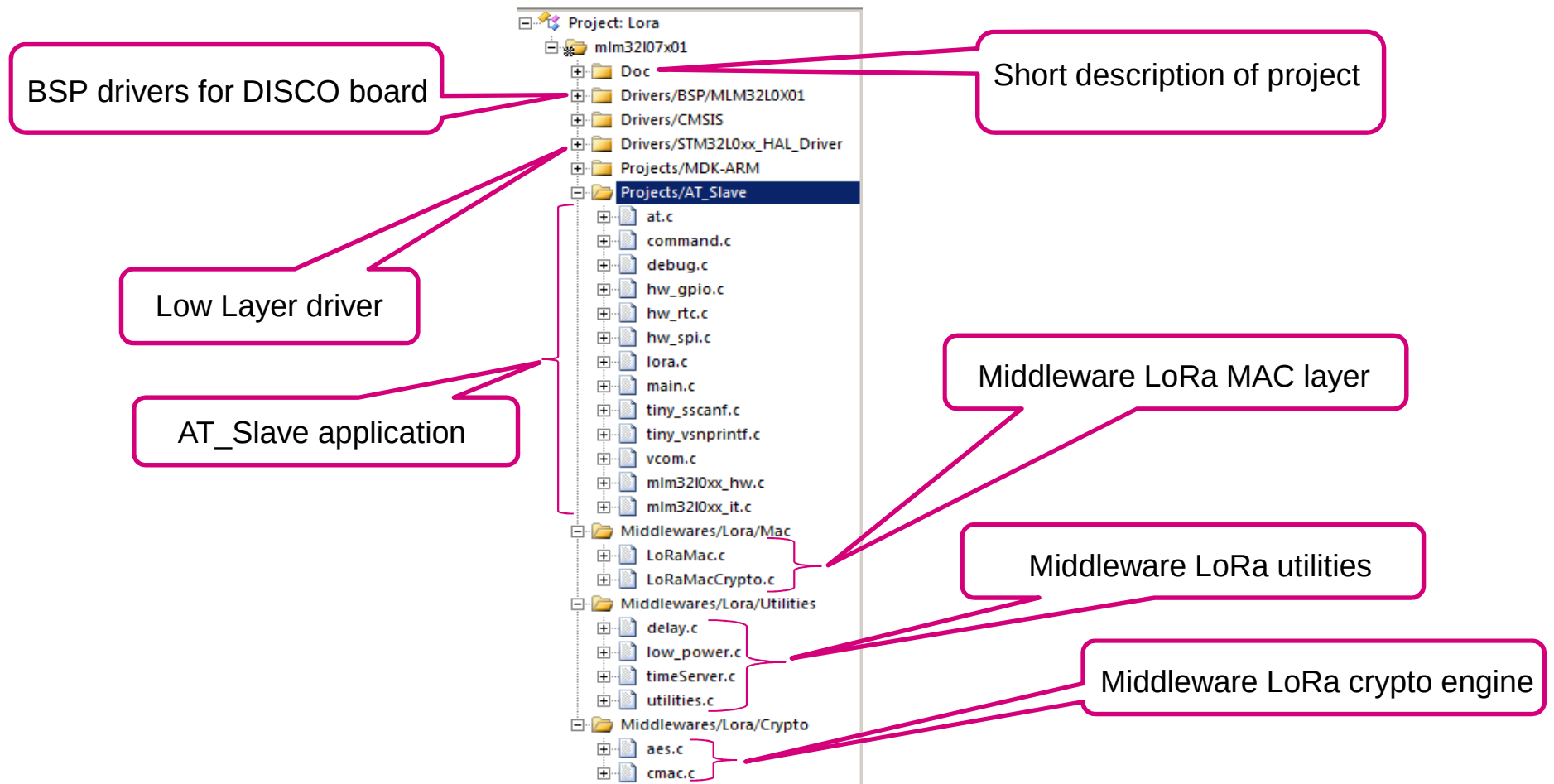


Hints:

Detailed description in UM2073: “STM32 LoRa® software expansion for STM32Cube”

Hands-on

LoRa “AT_Slave” project file structure



Hands-on

“AT_Slave” app files

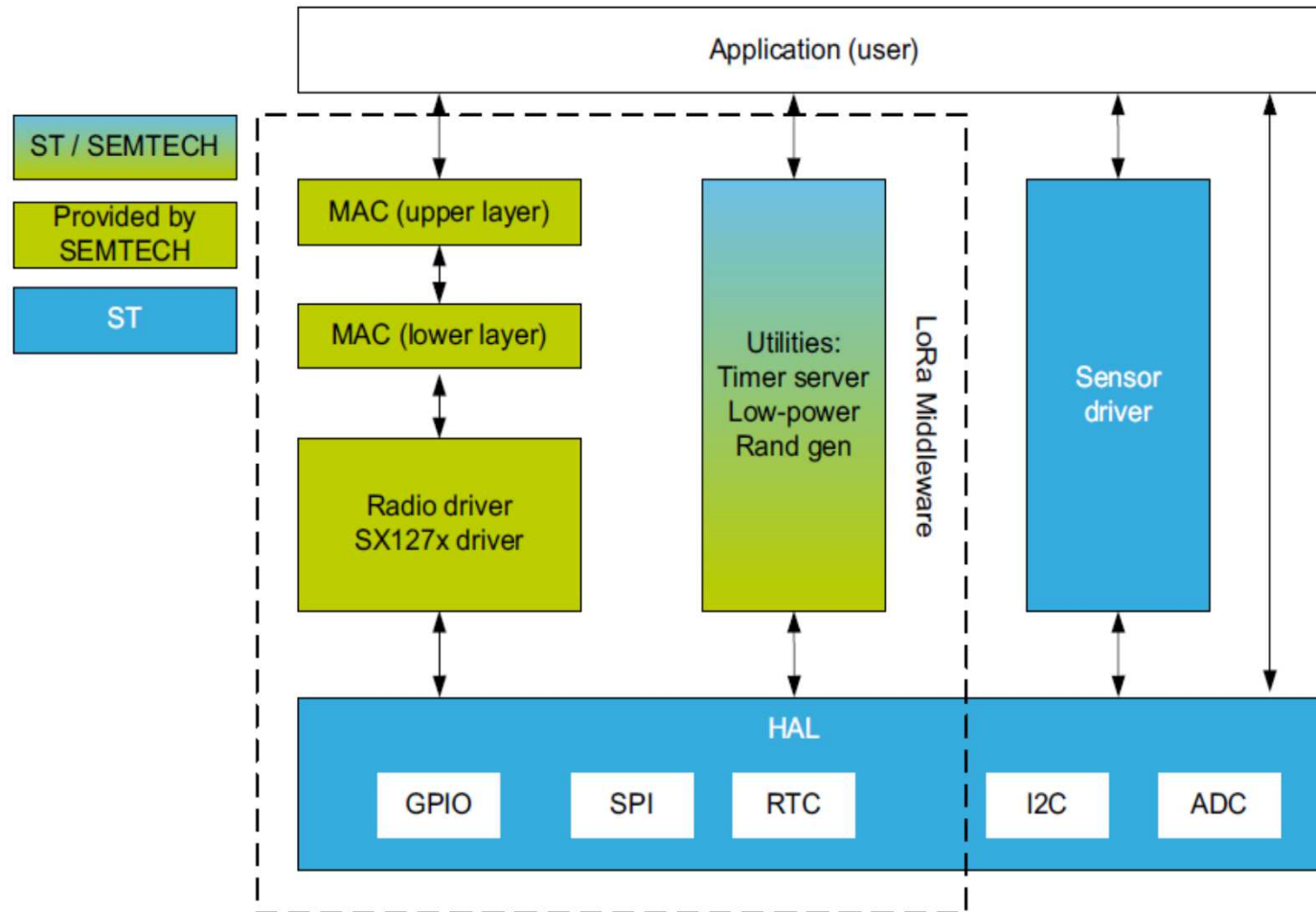
AT_Slave/Inc/at.h	Header for at.c
AT_Slave/Inc/Comissioning.h	End device comissioning parameters
AT_Slave/Inc/command.h	Header for command.c
AT_Slave/Inc/debug.h	Interface to debug functionally
AT_Slave/Inc/hw.h	Group all hw interface
AT_Slave/Inc/hw_conf.h	File to manage Cube SW family used and debug switch
AT_Slave/Inc/hw_gpio.h	Header for hw_gpio.c
AT_Slave/Inc/hw_msp.h	Header for driver hw msp module
AT_Slave/Inc/hw_rtc.h	Header for hw_rtc.c
AT_Slave/Inc/hw_spi.h	Header for hw_spi.c
AT_Slave/Inc/lora.h	Header for lora.c
AT_Slave/Inc/mlm32l0xx_hw_conf.h	Definitions for mlm32l0xx HW configuration
AT_Slave/Inc/mlm32l0xx_it.h	Header for mlm32l0xx_it.c
AT_Slave/Inc/stm32l0xx_hal_conf.h	Library Configuration file
AT_Slave/Inc/tiny_sscanf.h	Header for tiny_sscanf.c
AT_Slave/Inc/tiny_vsnprintf.h	Header for tiny_vsnprintf.c
AT_Slave/Inc/vcom.h	Interface to vcom.c
AT_Slave/Inc/version.h	Version file
AT_Slave/Src/at.c	AT commands API
AT_Slave/Src/command.c	Definitions of AT commands
AT_Slave/Src/debug.c	Debug driver
AT_Slave/Src/hw_gpio.c	Gpio driver
AT_Slave/Src/hw_rtc.c	Rtc driver
AT_Slave/Src/hw_spi.c	Spi driver
AT_Slave/Src/lora.c	LoRa API to drive the LoRa state machine
AT_Slave/Src/main.c	Main program file
AT_Slave/Src/mlm32l0xx_hw.c	MLM32l0xx specific hardware driver code
AT_Slave/Src/mlm32l0xx_it.c	MLM32l0xx Interrupt handlers
AT_Slave/Src/tiny_sscanf.c	Tiny implementation of sscanf() like function
AT_Slave/Src/tiny_vsnprintf.c	Tiny implementation of vsnprintf() like function
AT_Slave/Src/vcom.c	Virtual com port interface on Terminal

Hands-on

I-CUBE-LRWAN implementation basics

Hands-on

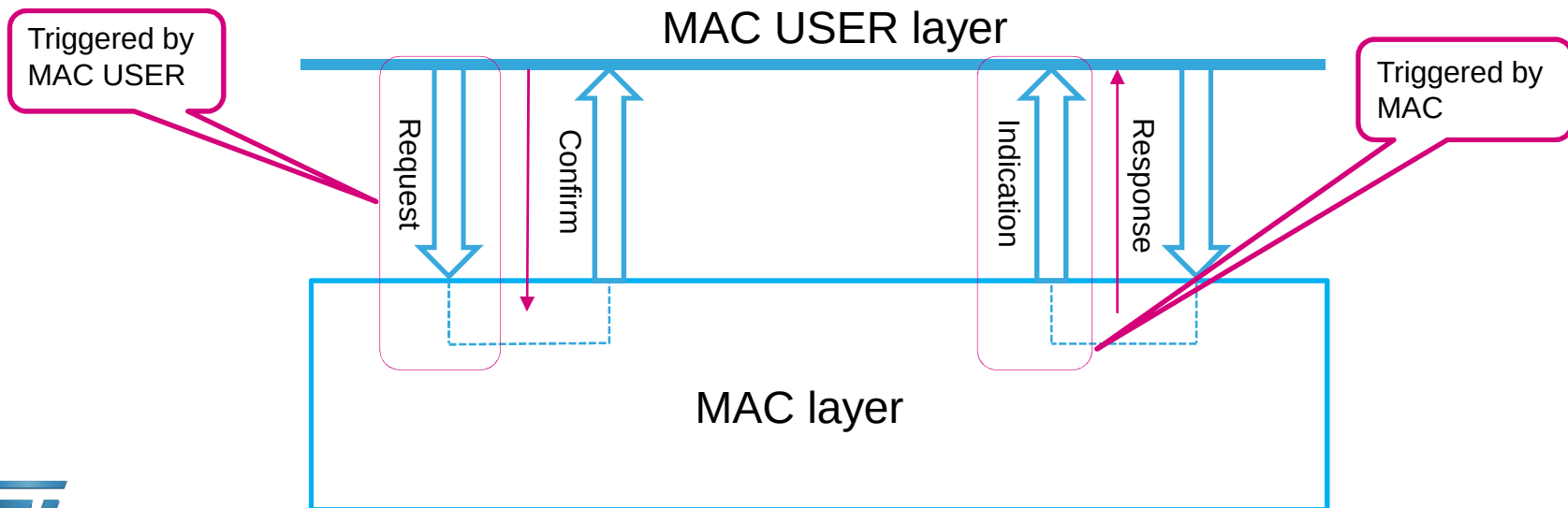
I-CUBE-LRWAN basics: software architecture



IEEE 802.15.4 introduction: MAC API

I-CUBE-LRWAN protocol stack implementation base on IEEE 802.15.4 standard. Communications are passed between the MAC layer and the next highest stack layer (MAC user) by means of “service primitives”

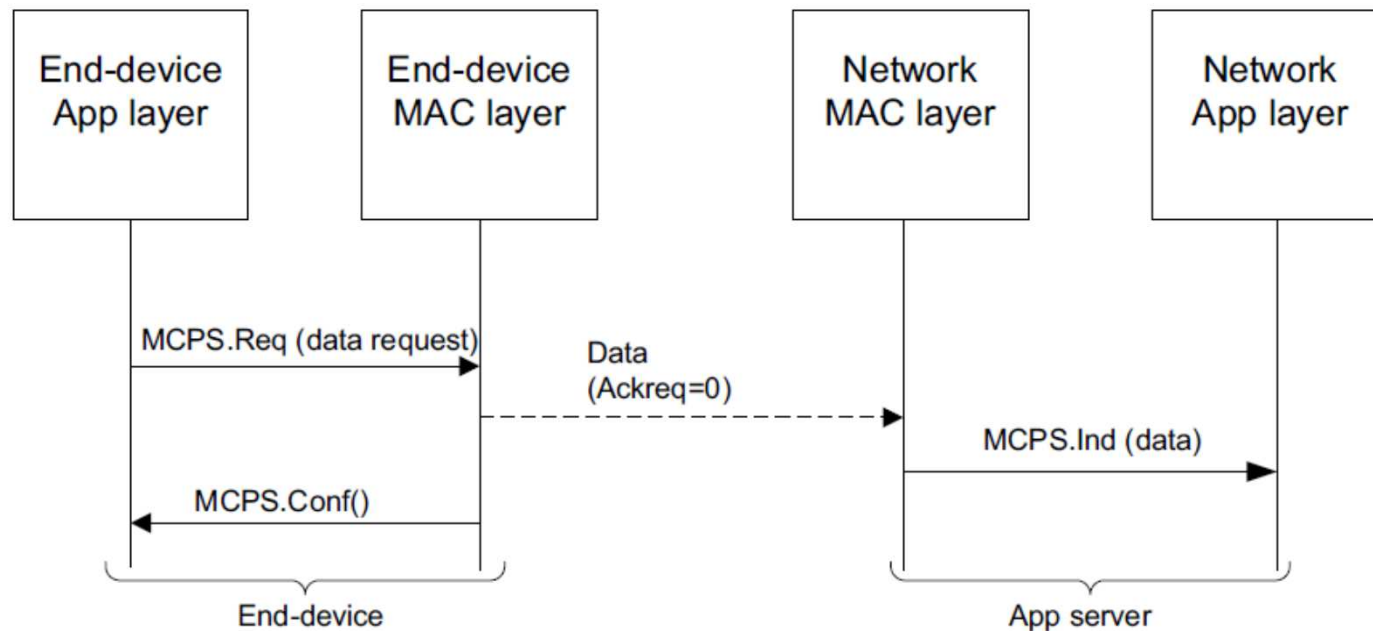
- Request ; initiated by the MAC user
- Confirm ; MAC user may solicit Confirm from the MAC layer
- Indication ; initiated by the MAC layer
- Response ; MAC layer may solicit Response from the MAC user



IEEE 802.15.4 introduction: MAC services

IEEE 802.15.4 MAC provides the interface to the application layer using two elements:

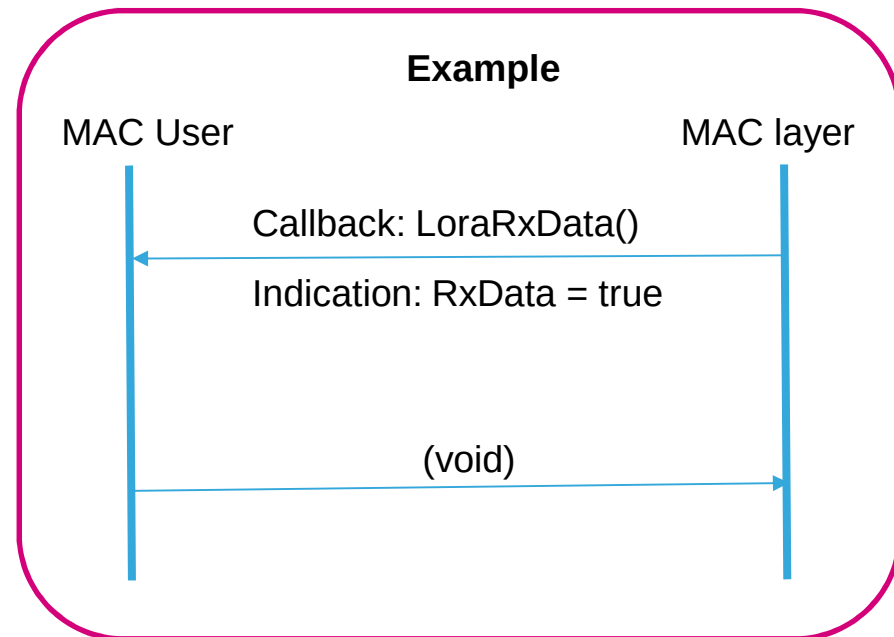
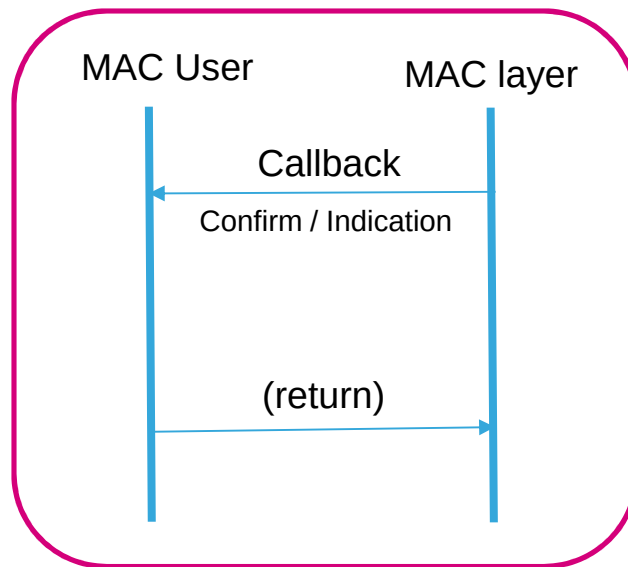
- MAC Management Service: this is called **MAC Layer Management Entity**, MLME. It provides the service through which layer management functions may be called or accessed. Example: JOIN network REQUEST,
- MAC Data Service: this is called the **MAC Common Port Service**, MCPS. It provides data transport services. Example: SEND data REQUEST.



Hands-on

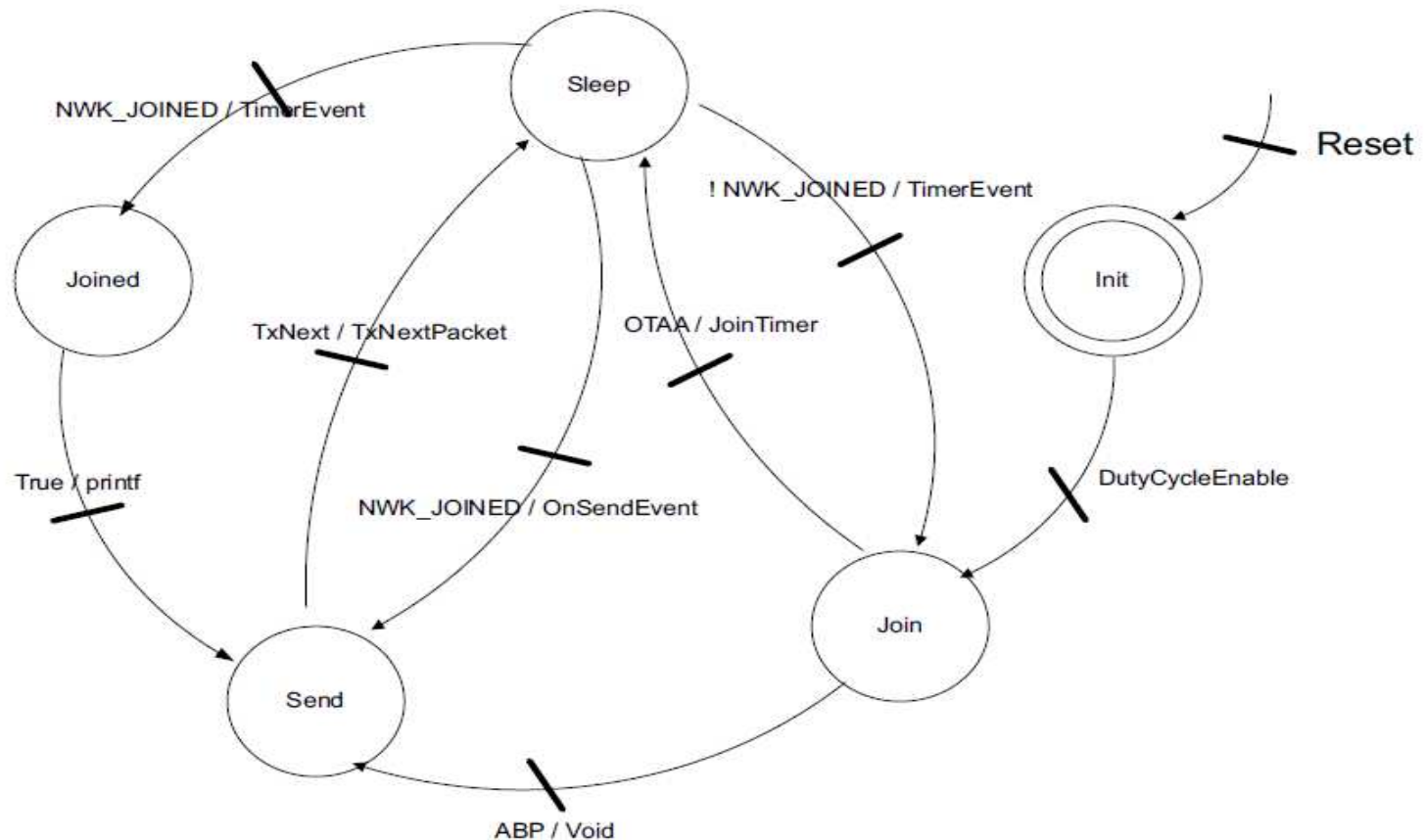
IEEE 802.15.4 introduction: callback mechanism

Request is issued by the MAC User by means of call to one of the MAC layer API functions. The most straightforward way for the MAC layer to reply (with Confirm and/or Indication) is via callback function.



I-CUBE-LRWAN basics: operation model

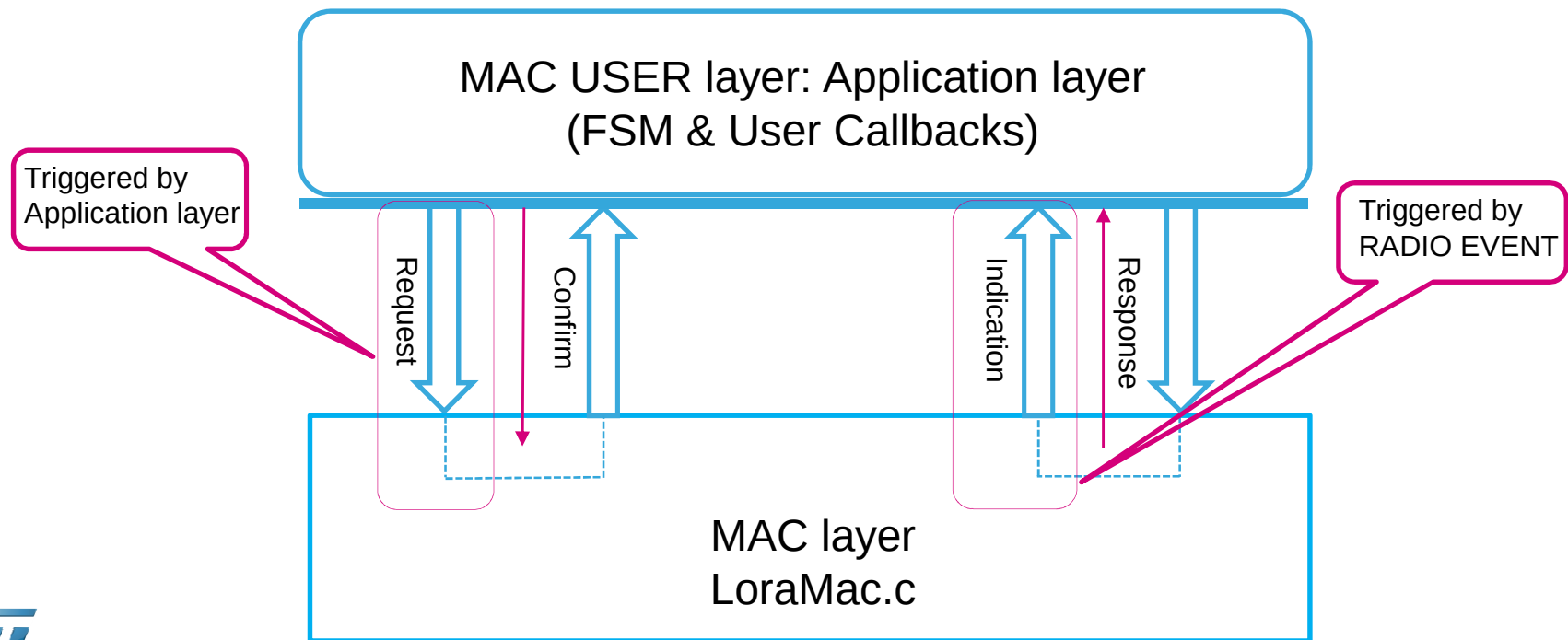
The model of operation proposed for this LoRa class A is based on **Finite State Machine (FSM)** running in the main thread of the application. LoRa FSM transition is triggered either by a timer event (Request) or by a radio event (Indication).



I-CUBE-LRWAN basics: operation model

IEEE 802.15.4 and operation model in practice:

- FSM requests services of MAC layer: Request / Confirm mechanism (i.e. send data)
- MAC layer indicates to FSM radio events and invokes related user callbacks (i.e. receive data event)



I-CUBE-LRWAN basics: Finite State Machine Hands-on

LoraWAN FSM is implemented in **lora.c** / **lora.h** source files

1. Initialization of the Lora class A FSM

```
void lora_Init (LoRaMainCallback_t *callbacks,  
LoRaParam_t* LoRaParamInit)
```

2. Lora class A FSM entry point

```
void lora_fsm (void)
```

3. Make Tx frame user callback function, user is free to implement own code here

```
void LoraTxData (lora_AppData_t *AppData,  
FunctionalState* IsTxConfirmed)
```

4. Make Rx frame user callback function, user is free to implement own code here

```
void LoraRxData (lora_AppData_t *AppData)
```

I-CUBE-LRWAN basics: Finite State Machine Hands-on

LoraWAN FSM is implemented in **lora.c** / **lora.h** source files

1. Get of the Lora class A FSM current state

```
DeviceState_t lora_getDeviceState( void );
```

Lora FSM states definition

```
typedef enum eDevicState  
{  
    DEVICE_STATE_INIT,  
    DEVICE_STATE_JOIN,  
    DEVICE_STATE_JOINED,  
    DEVICE_STATE_SEND,  
    DEVICE_STATE_CYCLE,  
    DEVICE_STATE_SLEEP  
} DeviceState_t;
```

I-CUBE-LRWAN basics: software timers

All the time dependences of I-CUBE-LRWAN are realized using software timer server **timeServer.c / timeServer.h**

1. Initialize the timer and associate callback

```
void TimerInit (TimerEvent_t *obj, void (*callback) (void))
```

2. Set the timer a timeout value on milliseconds

```
void TimerSetValue (TimerEvent_t *obj, uint32_t value)
```

3. Start the timer

```
void TimerStart (TimerEvent_t *obj)
```

4. Stop the timer

```
void TimerStop (TimerEvent_t *obj)
```

Hints:

User defined software timer can be used i.e. to control Tx and/or Rx led

I-CUBE-LRWAN basics: main loop

```
while (1)
{
    /* run the LoRa class A state machine*/
    lora_fsm();

    /* Handle UART commands */
    CMD_Process();

    /*
     * low power section
     */
    DISABLE_IRQ();
    /*
     * if an interrupt has occurred after DISABLE_IRQ, it is kept pending
     * and cortex will not enter low power anyway
     * don't go in low power mode if we just received a char
     */
    if ((lora_getDeviceState() == DEVICE_STATE_SLEEP) && (IsNewCharReceived() == RESET))
    {
#ifdef LOW_POWER_DISABLE
        LowPower_Handler();
#endif
    }
    ENABLE_IRQ();

    /* USER CODE BEGIN 2 */
    /* USER CODE END 2 */
}
```

FSM call

AT commands process call

I-CUBE-LRWAN basics: mlm32l0xx hw functions

There are functions related to hardware to speed up development process implemented in **mlm32l0xx_hw.c / hw_msp.h**

1. Get value of internal temperature sensor of STM32L072 in [°C]

```
uint16_t HW_GetTemperatureLevel( void );
```

2. Get battery level, 1 (very low) to 254 (fully charged)

```
uint8_t HW_GetBatteryLevel( void );
```

3. Low power modes management

4. Get board 64 bits unique ID

5. Peripherals management

Hands-on

I-CUBE-LRWAN basics: stack parametrization

Parametrization in file **Commissioning.h**

```
/* APB or OTAA activation */
#define OVER_THE_AIR_ACTIVATION      1

/* Private or public network connection */
#define LORAWAN_PUBLIC_NETWORK      true

/* DEVICE_EUI generated automatically or equal to LORAWAN_DEVICE_EUI */
#define STATIC_DEVICE_EUI            0
#define LORAWAN_DEVICE_EUI          { 0x01, 0x01, 0x01, 0x01, 0x01, 0x01, 0x01, 0x01 }

/* Application IEEE EUI */
#define LORAWAN_APPLICATION_EUI      { 0x01, 0x01, 0x01, 0x01, 0x01, 0x01, 0x01, 0x01 }

/* AES encryption/decryption cipher application key */
#define LORAWAN_APPLICATION_KEY      { 0x2B, 0x7E, 0x15, 0x16, 0x28, 0xAE, 0xD2, 0xA6, 0xAB, 0xF7, 0x15,
0x88, 0x09, 0xCF, 0x4F, 0x3C }

/* Current network ID */
#define LORAWAN_NETWORK_ID           ( uint32_t )0

/* DEVICE ADDRESS is generated using random generator or equal to LORAWAN_DEVICE_ADDRESS */
#define STATIC_DEVICE_ADDRESS        0

#define LORAWAN_DEVICE_ADDRESS       ( uint32_t )0x0100000a
/* AES encryption/decryption cipher network session key */
#define LORAWAN_NWKSKEY              { 0x2B, 0x7E, 0x15, 0x16, 0x28, 0xAE, 0xD2, 0xA6, 0xAB, 0xF7,
0x15, 0x88, 0x09, 0xCF, 0x4F, 0x3C }

/* AES encryption/decryption cipher application session key */
#define LORAWAN_APPSKEY              { 0x2B, 0x7E, 0x15, 0x16, 0x28, 0xAE, 0xD2, 0xA6, 0xAB,
0xF7,
0x15, 0x88, 0x09, 0xCF, 0x4F, 0x3C }
```

Hands-on

I-CUBE-LRWAN basics: stack parametrization

Parametrization in file **main.c**

1. Set definition of application data transmission duty cycle i.e. 10s

```
#define APP_TX_DUTYCYCLE    10000
```

2. LoraWAN Adaptive Data Rate active

```
#define LORAWAN_ADR_ON      1
```

3. LoraWAN confirmed messages OFF

```
#define LORAWAN_CONFIRMED_MSG    DISABLE
```

4. LoraWAN application port

```
#define LORAWAN_APP_PORT      2
```

Hands-on

I-CUBE-LRWAN basics: stack parametrization

1. Set duty cycle control in file **lora.c**
`#define LORAWAN_DUTYCYCLE_ON true`
2. Set join request trials duty cycle in file **lora.c** [ms]
`#define OVER_THE_AIR_ACTIVATION_DUTYCYCLE 10000`

Hints:

According to EN300220-1 there is a duty cycle limitation in ISM band: “*In a period of 1 hour the duty cycle shall not exceed the spectrum access and mitigation requirement values...*”. For 868MHz sub-band used by Lora it is 1%. LoraWAN protocol stack controls duty cycle but for our hands-on we can switch it off due to practical reasons (quite high on-air traffic).

I-CUBE-LRWAN basics: debug

PRINTF stream redirection to ST-Link Virtual Com Port was implemented in files **vcom.c** / **vcom.h**

1. Initialization of LPUART, default parameters: 9600,8,N,1

```
void vcom_Init(void)
```

```
LPUART_InitStruct.BaudRate = 9600;  
LPUART_InitStruct.DataWidth = LL_LPUART_DATAWIDTH_8B;  
LPUART_InitStruct.StopBits = LL_LPUART_STOPBITS_1;  
LPUART_InitStruct.Parity = LL_LPUART_PARITY_NONE;  
LPUART_InitStruct.TransferDirection = LL_LPUART_DIRECTION_TX_RX;  
LPUART_InitStruct.HardwareFlowControl = LL_LPUART_HWCONTROL_NONE;
```

2. Redefined PRINTF() function can be used instead of printf() to output debug stream to Com Port terminal i.e.

```
PRINTF("Attention command interface\n\r");
```

Hints: LPUART wakes up MCU from STOP mode and due to wake up time of LPUART clock source (HSI), high transmission bit rates cannot be reached.

I-CUBE-LRWAN basics: debug

1. Enable debug GPIO pins: GPIOB13/14/15 in file **hw_conf.h** to observe time of MCU activity:
 - PB13: wakeup time calculation,
 - PB14: SLEEP mode,
 - PB15: STOP mode

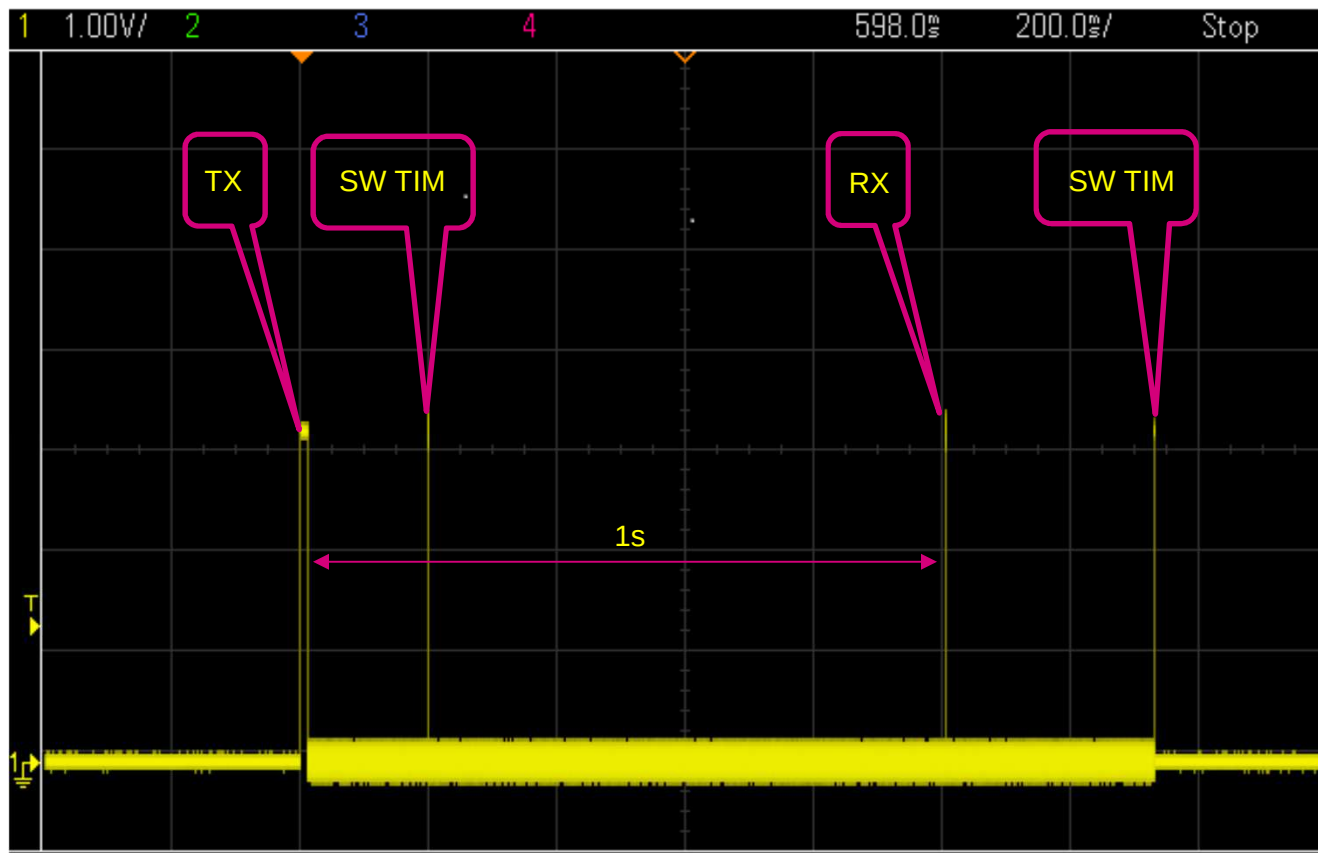
```
//#define DEBUG
```
2. Enable trace functionality in file **hw_conf.h** to output debug data via DBG_PRINTF macro to terminal (high talkative level)

```
//#define TRACE  
DBG_PRINTF("RX on freq %d Hz at DR %d\n\r", frequency, dr);
```

Hands-on

I-CUBE-LRWAN basics: GPIO debug examples

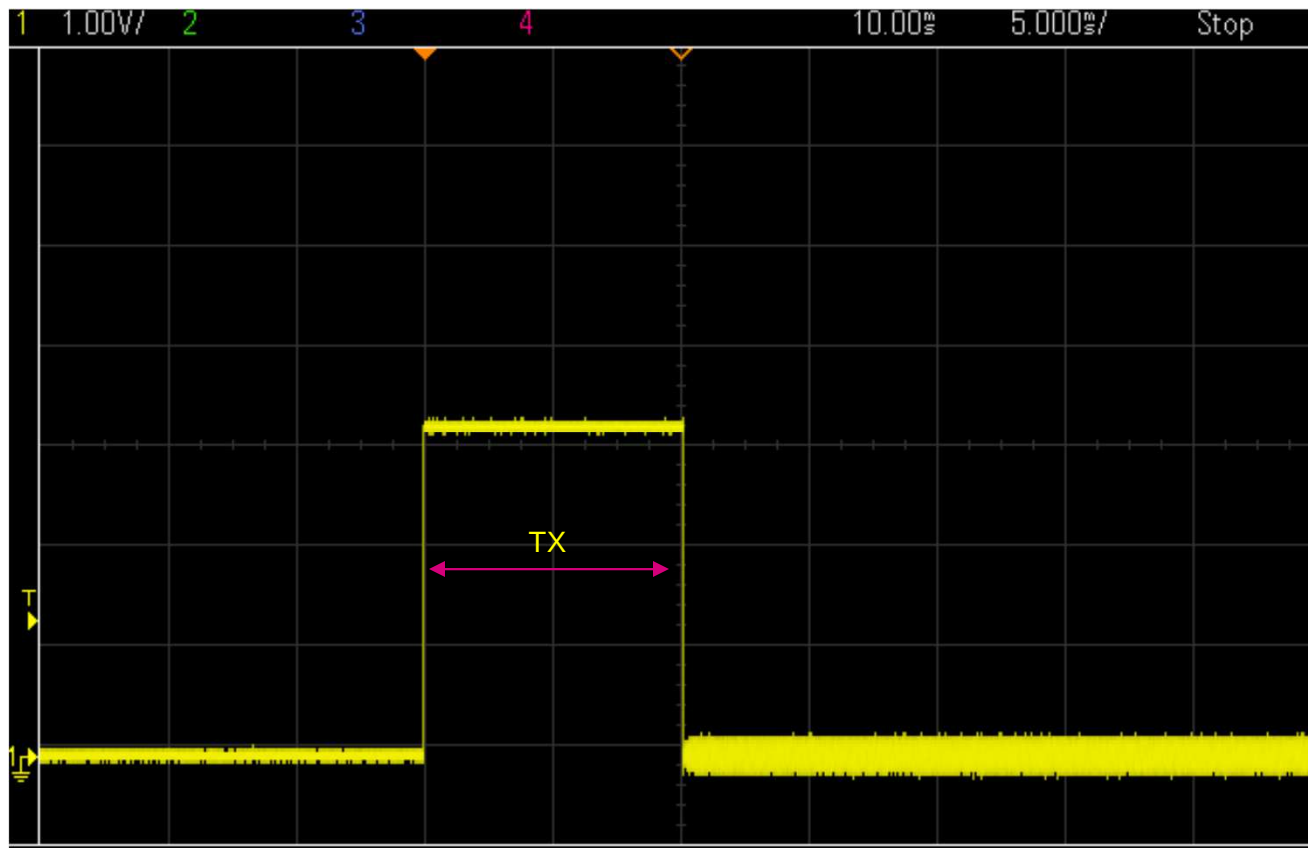
PB15: STOP mode activity monitoring



Hands-on

I-CUBE-LRWAN basics: GPIO debug examples

PB15: sending 2-bytes long payload (MCU RUN mode)



Hands-on

I-CUBE-LRWAN basics: low power mode

1. Disable low power mode of application in file **hw_conf.h**
`//#define LOW_POWER_DISABLE`

Hands-on

Thank you